

CardioView

Clinical Echocardiogram Diagnostics

User Manual

Complete guide to the AI-powered clinical interface

Version: 1.0 | Platform: Flet 0.84 (Python) |
Architecture: MAE ViT

Detection of cardiac pathologies
via deep learning

Document generated automatically
June 16, 2026

Contents

1	Introduction	2
1.1	What is CardioView?	2
1.2	General architecture	2
1.3	Prerequisites	2
1.4	Connection mode: cluster or local	3
2	Title bar and global settings	4
2.1	Language switching	4
3	Main header	5
4	Tab 1 — Load & Preprocessing	6
4.1	Step bar	6
4.2	Species selector	6
4.3	DICOM load area	7
4.4	Interactive preview	7
4.5	Preprocessing configuration	7
4.5.1	Left column — Analysis type	8
4.5.2	Right column — Parameters per step	8
4.5.3	Dialog buttons	11
4.6	Running the preprocessing	11
5	Tab 2 — Diagnosis	13
5.1	Left panel — Model selection	13
5.1.1	Active model	13
5.1.2	Diagnosis dataset	13
5.2	Right panel — Inference configuration	14
5.2.1	Output files (diagnosis)	14
5.3	Results visualisation — Diagnosis by patient	14
5.4	Computational results	15
5.5	Evaluation section — Validate the model	15
5.5.1	Cluster validation and automatic retraining	16
6	Tab 3 — Prognosis (mouse only)	18
6.1	Prognosis model selection	18
6.2	Configuration and execution	18
6.2.1	Output files (prognosis)	18

6.3	Results visualisation — Prognosis	19
7	Complete workflow	20
7.1	Diagnosis workflow — Human	20
7.2	Complete workflow — Mouse (Diagnosis + Prognosis)	21
8	Key differences between human and mouse	22
9	Frequently asked questions and troubleshooting	23
	Glossary	24

Chapter 1

Introduction

1.1 What is CardioView?

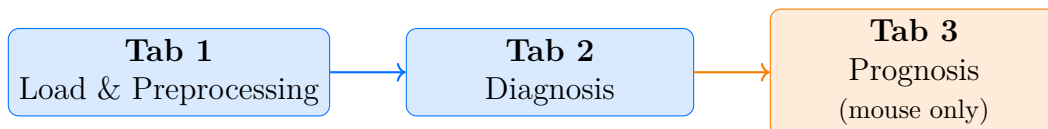
CardioView is a clinical desktop interface developed in Python with the **Flet 0.84** framework (Flutter under the hood). Its purpose is to assist medical professionals in the **detection and classification of cardiac pathologies** from echocardiography videos in DICOM format, using deep learning models based on the **Masked Autoencoder Vision Transformer (MAE ViT)** architecture.

The tool supports two parallel workflows:

- **Human echocardiography** — diagnosis of pathologies among the classes HIPER (hypertrophy), ISQ (ischaemia) and NORM (normal).
- **Mouse echocardiography** — diagnosis among the classes CTL (control), DB (diabetic), OB (obese) and SAH; plus a **cardiac failure prognosis** module (estimated time in months).

1.2 General architecture

The interface is organised into **three main tabs** accessible from the top bar:



The logical flow is: load video → preprocess → run diagnosis model → (optionally) run prognosis model.

1.3 Prerequisites

- Python ≥ 3.10 with the virtual environment activated.
- Packages: `flet`, `pydicom`, `opencv-python`, `torch`, `numpy`, `pandas`, `Pillow`, `torchinfo`.

- For HODMD: DMDd and hosvd modules in the preprocessing directory.
- Input DICOM video (.dcm or .dicom).

Starting the application

Run in terminal:

```
python main.py
```

The window starts at 1280×840 px (minimum 920×640 px) with a custom title bar.

1.4 Connection mode: cluster or local

Before the main interface loads, a **login screen** offers two options:

Option	Behaviour
Connect to cluster	With the research cluster’s username and password, the app enables connected mode : pre-processed data is automatically uploaded to the server, clinician validations are recorded on the cluster, and the cluster can notify and launch automatic model retraining (see Section 5.5.1).
Continue offline	Opens the interface in local mode , intended for sensitive data that must not leave the machine. No data is sent to the cluster: preprocessing, diagnosis, prognosis and validation all run entirely locally.

Switching modes

The connection status is shown in the header (cluster username or «Offline», with a green/grey indicator). From the avatar menu you can log in (local → connected) or log out (connected → local) at any time.

Chapter 2

Title bar and global settings

The title bar is fully custom (the native OS bar is hidden). It contains, from left to right:

Element	Description
Icon + title	Heart logo and the text <i>CardioView — Clinical Diagnostics</i> . Acts as a drag area to move the window.
Settings button	Classic-style drop-down menu with two entries: <i>User manual</i> (no action yet) and <i>Language</i> (Spanish / English). When the language changes the entire interface is rebuilt instantly.
—	Minimise window.
□	Maximise / restore window.
×	Close the application.

2.1 Language switching

Clicking **Settings** → **Language** shows the options *Spanish* and *English*. A tick mark (✓) indicates the active language.

Chapter 3

Main header

Below the title bar the application **header** is displayed with a dark-blue gradient background. Its elements are:

- **Logo** — heart icon on a blue background with a glow shadow.
- **Title** — «Detection of cardiac [pathologies](#)» (the cyan term changes to [pathologies](#) in English and «[cardíacas](#)» in Spanish).
- **Subtitle** — «Echocardiogram analysis · Clinical AI».
- **User panel** — shows the connected cluster username (or «Offline» in local mode) next to a status indicator (green = connected, grey = local) and an avatar with the user's initials. The avatar menu logs in or out.

Chapter 4

Tab 1 — Load & Preprocessing

This tab is the **entry point** of the workflow. It is split into two columns: left (load and preview) and right (preprocessing). A step bar at the top summarises the process status.

4.1 Step bar

The bar shows four numbered circular indicators:

1. **Load DICOM** — active until a file is selected.
2. **Verify video** — active once the DICOM is loaded.
3. **Configure preprocessing** — active until the preprocessing configuration is saved.
4. **Ready to analyse** — active once preprocessing has been executed.

A horizontal connector between indicators turns blue when the previous step is complete.

4.2 Species selector

Mandatory step — select species before loading

This selector **must be configured before any other action**. It determines which models, preprocessing scripts and diagnostic classes will be used throughout the session.

The red-bordered card contains two large buttons:

Button	Diagnostic classes	Available modules
Human ()	HIPER — Hypertrophy ISQ — Ischaemia NORM — Normal	Diagnosis

Button	Diagnostic classes	Available modules
Mouse ()	CTL — Control DB — Diabetic OB — Obese SAH — Subarachnoid haemorrhage	Diagnosis + Prognosis

4.3 DICOM load area

- «**Select DICOM Video**» button — opens the native OS file dialog accepting any extension. Both `.dcm` and `.dicom` are supported.
- Once a file is selected, its name, size (KB or MB) and a green «Loaded successfully» chip appear in the same card.

DICOM compression

If the file uses JPEG, JPEG-LS or JPEG-2000 compression, `Pillow` and `pylibjpeg` are required. A descriptive error message indicates which package to install.

4.4 Interactive preview

After decoding, a preview card appears with:

Control	Function
DICOM viewer	Current frame image (max. 512 px, JPEG 75%).
 / 	Go to previous / next frame. Disabled for single-frame DICOMs.
 / 	Play or pause the animation at 10 fps. Playback uses Flet's event loop (<code>page.run_task</code>) to avoid blocking the UI.
Slider	Navigate to any frame by position.
Frame counter	«Frame X / N» updated in real time.
Metadata	Table with: File, Size, Frames, Full path, Modality (DICOM Video), Status (Ready ✓).

4.5 Preprocessing configuration

This section appears in the right column **only after a file has been loaded**. Initially it shows the «Not configured» status with four informational chips (Video only, SVD, HODMD, SVD+HODMD).

The «**Configure Preprocessing**» button opens a modal dialog with two columns:

4.5.1 Left column — Analysis type

A radio group with four options:

Option	Steps executed	Description
Original video	Step 1	Frame extraction and cleaning. Common to all pipelines.
Original video + SVD	Steps 1–2	Adds Singular Value Decomposition reconstruction. Filters structural noise.
Video + SVD + HODMD	Steps 1–3	Adds HODMD dynamics. Captures relevant temporal modes.
Video + SVD + HODMD + SVD	Steps 1–4	Applies a second SVD on HODMD modes (maximum feature extraction).

4.5.2 Right column — Parameters per step

The right panel changes dynamically according to the analysis type and the selected species.

Step 1 · Video preprocessing

Human:

Parameter	Default	Effect
Remove white lines	On	Inpainting over horizontal white lines from the ultrasound equipment.
Second inpainting	Off	Additional inpainting pass (experimental).
Remove colour high-lights	Off	Suppresses saturated colour artefacts.
Crop to square image	On	Crops the useful echocardiography area to a square.
Output directory	(empty)	If empty, the folder <code>preprocess_<video_name></code> is created next to the DICOM.

Mouse:

Mouse-specific parameters — Step 1

Mouse parameters differ from human ones because rodent echocardiographies have distinct characteristics (higher heart rate, lower spatial resolution, on-screen text overlay).

Parameter	Default	Effect
# Snapshots	(empty = all)	Limits the number of frames to process. Useful for quick tests.
Colour map	gray	Colour space for saving frames (gray , jet ...).
Colour inpainting	Off	Corrects colour artefacts in mouse images (experimental).
Letter recognition	Off	Detects and removes text overlaid on the video (experimental). Activates the threshold sub-panel:
Lower threshold	210	Minimum threshold (0–255 scale) for white text detection.
Upper threshold	220	Maximum threshold.
Export data (.npy)	On	Saves processed frames as NumPy arrays for subsequent steps.
Read and display DICOM info	Off	Prints DICOM file meta-data to the console.

The output folder for mouse is named `mice_preprocess_<video_name>` while for humans it is `preprocess_<video_name>`.

Step 2 · SVD reconstruction

Available when choosing *Original video + SVD*, *Video + SVD + HODMD* or *Video + SVD + HODMD + SVD*.

SVD decomposes the frame sequence as a matrix and retains only the k dominant modes, removing incoherent noise:

$$\mathbf{X} \approx \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top$$

Parameter	Default	Effect
Number of SVD modes	5	How many singular vectors (modes) to retain. More modes = more detail, but also more noise.
Export SVD images and data	On	Saves reconstructed images and SVD arrays.
Export V matrices	Off	Also saves the temporal modes $\mathbf{V}$.
Colour map	gray	Export colourmap.
Computation library	NumPy (CPU)	NumPy (CPU) or PyTorch (GPU). Use GPU if CUDA is available.
Export format	.npy	.npy (NumPy), .mat (MATLAB) or no export.

Reconstructed data are saved to:

```
<preprocess_dir>/results_step2_svd/svd_reconst_data/
```

Step 3 · HODMD reconstruction

Available in *Video + SVD + HODMD* and *Video + SVD + HODMD + SVD*.

HODMD (Higher-Order Dynamic Mode Decomposition) extends classical DMD to a higher-order state space via iterative HOSVD + DMD, capturing multi-frequency dynamic modes:

Parameter	Default	Effect
ε_1 — SVD tolerance	0.0005	Truncation threshold of the internal SVD step. Lower = more modes retained.
ε_2 — DMD tolerance	0.0005	Truncation threshold of the DMD step.
Parameter d	/3	Order of the augmented state. /3 = one third of total frames; /5 = one fifth; positive integer = fixed value.
deltaT (time step)	0.004	Time interval between frames (seconds).
Analyse complex data	Off	Treats the field as complex. Required for phase signals.

Parameter	Default	Effect
Computation library (HODMD)	NumPy	NumPy, Hybrid (recommended for speed) or PyTorch (GPU).

Results are saved to:

```
<preprocess_dir>/results_step3_hodmd/DMD_solution/
reconstructed_images_data/
```

HODMD dependencies

Step 3 requires the `DMDd` and `hosvd` modules. If they are not available, an error message is shown when attempting to execute.

Step 4 · SVD on HODMD modes

Only available in *Video + SVD + HODMD + SVD*. Applies a second SVD on the images reconstructed by HODMD to extract the most compact and discriminative modes.

The parameters are identical to those of Step 2 (number of modes, library, export format). Results go to:

```
<preprocess_dir>/results_step4_svd/abs/svd_reconst_data/
```

4.5.3 Dialog buttons

Button	Action
Cancel	Closes the dialog without saving changes.
Save configuration	Validates and stores all parameters in <code>state["params"]</code> , closes the dialog and shows a summary in the preprocessing card.

4.6 Running the preprocessing

Once the configuration is saved, the preprocessing card shows:

- The selected analysis type and key parameters of each step in informational chips.
- A «**Run Preprocessing**» button (green gradient).
- A «**Reconfigure**» button to reopen the dialog.

When **Run** is pressed:

1. The main thread launches each step asynchronously with `asyncio.to_thread` to avoid blocking the interface.

2. A loading spinner indicates «Running step N...» in real time.
3. If an error occurs in any step, a red block shows the full exception message.
4. On success, a green block shows: *Frames processed*, *Shape*, *Pipeline (mice/humans)*, *Output directory*.
5. The Diagnosis and Prognosis tabs are notified to automatically update their data paths.

Generated folder structure

```
preprocess_<video>/          (or mice_preprocess_<video>/)
  results_step1_raw/
    original_data/
      <sequence>--original--data/
        frame_0000.npy    ...
  results_step2_svd/
    svd_reconst_data/    ...
  results_step3_hodmd/
    DMD_solution/
      reconstructed_images_data/    ...
  results_step4_svd/
    abs/svd_reconst_data/    ...
```

Chapter 5

Tab 2 — Diagnosis

The diagnosis tab allows you to **select a MAE ViT model**, **run inference** on the preprocessed data and **visualise classification results**.

It is organised into two columns: left panel (model and dataset selection) and right panel (paths, device and run button). Below these panels the results sections are displayed.

5.1 Left panel — Model selection

5.1.1 Active model

At startup the system **automatically scans** the models folder and shows the models consistent with the preprocessing. The scan is repeated every time the preprocessing level or species changes.

5.1.2 Diagnosis dataset

Appears only when preprocessing has been executed. Allows choosing which representation to use as model input:

Dataset	Relative path within preprocess_<video>/
Original data (Step 1)	results_step1_raw/original_data/
SVD Reconstr. (Step 2)	results_step2_svd/svd_reconst_data/
HODMD Reconstr. (Step 3)	results_step3_hodmd/DMD_solution/reconstructed_images_da
SVD on HODMD modes (Step 4)	results_step4_svd/abs/svd_reconst_data/

The system automatically marks the ★ **Recommended** dataset based on the preprocessing level (HODMD if available, original otherwise).

5.2 Right panel — Inference configuration

Field	Description
Preprocessed data path	Auto-filled when selecting a dataset. Can also be browsed manually with the Browse... button.
Results directory	Auto-generated as <code><data>/../diagnosis_resultados_<model>-<timestamp></code> (prefix <code>mice_diagnosis_</code> for mice).
Compute device	CPU (default), CUDA (GPU), CUDA:0, CUDA:1.
Run Model button	Launches <code>run_inference.py</code> as a subprocess with a 60-minute timeout.

5.2.1 Output files (diagnosis)

```
<output_dir>/
diagnosis_resultados_<model>-<timestamp>/    (or mice_diagnosis_
...)
input_parameters.txt          -- parameters of the model used
test_prediction_stats.csv     -- prediction per frame
test_seq_prediction_stats.csv -- prediction per sequence
computational_results.csv     -- times and model metrics
```

Column	Description
<code>secuencia</code>	Sub-folder name (patient/sequence).
<code>pred_clase_media</code>	Class with the highest mean confidence.
<code>conf_media_CLASS</code>	Mean confidence (0–1) for each class.
<code>frames_CLASS</code>	Number of frames predicted as each class.

5.3 Results visualisation — Diagnosis by patient

After inference (or when loading a results folder manually), a card is built per sequence showing:

Element	Description
Metadata	Patient (first part of the name before -), data type (second part) and model used.
Predicted diagnosis	The class with the highest confidence is displayed large with its diagnostic colour and a glowing border.

Element	Description
Confidence bars	One bar per class (width proportional to mean confidence). The winning class is highlighted with a green background.
Frame chips	For each class: how many frames were assigned to it.

Diagnostic colours:

Species	Class	Colour
Human	HIPER	 Red
	ISQ	 Orange
	NORM	 Green
Mouse	CTL	 Green
	DB	 Orange
	OB	 Blue
	SAH	 Red

5.4 Computational results

A separate card shows model performance metrics:

Metric	Description
Total frames	Total .npy files processed.
Total time	Inference time in seconds.
Avg / frame	Average time per frame in milliseconds.
Throughput	Images processed per second.
Parameters	Millions of trainable model parameters.
Model size	Estimated size in MB (weights + activations).

An **execution context** block also shows: model used, data used and preprocessing level.

5.5 Evaluation section — Validate the model

Purpose

This section allows the clinician to specify the **true class** of the patient, generate a **partial confusion matrix** with performance metrics, and — in connected mode — **validate** that result for the model's retraining process.

The user can:

1. Select «Validate the model».
2. Choose the true class via a radio group (colour-coded by class) or type it manually in the text field («Other class. . . »).
3. Adjust the **clinician confidence** (0–100%) with the slider or numeric field.
4. Press «**Generate matrix**» (local mode) or «**Generate matrix and validate**» (connected mode).

The matrix shows predicted classes in rows and true classes in columns, with a green background on the diagonal (correct) and red off-diagonal (incorrect). Metrics computed below:

Metric	Formula
Accuracy	correct/total frames
Precision	1 if there are correct predictions, 0 otherwise
Recall	Equal to Accuracy (single-label case)
F1-Score	$2 \cdot P \cdot R / (P + R)$

The option «Do not validate the model» disables the entire section.

5.5.1 Cluster validation and automatic retraining

Connected mode only

In **local mode**, «Generate matrix» computes the confusion matrix on the machine only: nothing is sent to or received from the cluster.

If the session is **connected to the cluster**, pressing «Generate matrix and validate» also sends the indicated **true class** and the **clinician confidence** to the server. This validated label is added to the model’s retraining dataset.

Automatic retraining after N videos. The cluster counts how many valid samples have accumulated for the current species and task (diagnosis or prognosis). Once the threshold (**5 videos**) is reached, the interface shows a «Retraining pending» notice with a 5-minute countdown and two options:

- **Validate now** — closes the notice without stopping the countdown, giving the clinician time to review or correct the class of the current video from this same section.
- **No, retrain now** — launches retraining immediately, without waiting for the countdown to finish.

When the countdown ends (or is forced), the cluster rebuilds the retraining dataset with the latest validated labels and launches training on its GPUs; a notification confirms whether the process started successfully.

Incomplete classes

If validated videos for some required class are still missing, retraining is automatically postponed until all classes are represented, and re-checked on the next cycle.

Chapter 6

Tab 3 — Prognosis (mouse only)

Module exclusive to mouse echocardiography

The Prognosis tab **only operates with mouse echocardiography**. If used with the «Human» species, an orange warning is shown. Prognosis models are located in `execute_models/` and have been validated exclusively on rodent data.

The prognosis module estimates the **time to cardiac failure** in months, using a MAE ViT regression model (output layer with 1 neuron).

6.1 Prognosis model selection

The interface is identical to that of the diagnosis tab (same scan mechanism, model cards, compact display).

6.2 Configuration and execution

Field	Description
Preprocessed data path	Auto-filled with the mouse data path if pre-processing was done in the same session.
Results directory	Prefix <code>mice_prognosis_resultados-<timestamp></code> .
Device	CPU / CUDA.
Run Model	Launches <code>run_inference_prognosis.py</code> .

6.2.1 Output files (prognosis)

```
mice_prognosis_resultados-<timestamp>/
input_parameters.txt
test_prediction_stats.csv      -- outcome_time_estimated per
    frame
test_seq_prediction_stats.csv -- statistics per sequence
```

```
computational_results.csv -- times and metrics
```

6.3 Results visualisation — Prognosis

For each sequence a card is shown with:

Element	Description
Patient	First part of the sequence name.
Estimated prognosis	Value in months displayed prominently in cyan. If it exceeds 12 months, it is converted to years (e.g. «1 year and 3 months»).
Min–max range	Minimum and maximum of the frame-by-frame prediction.
Standard deviation	Variability of the prediction across frames.
Number of frames	Frames processed.

The section header displays: «**ESTIMATED TIME TO CARDIAC FAILURE**» on a cyan bar.

Clinical warning

The prognosis module has been validated exclusively on mouse echocardiography. Results must not be used as a substitute for human clinical diagnosis.

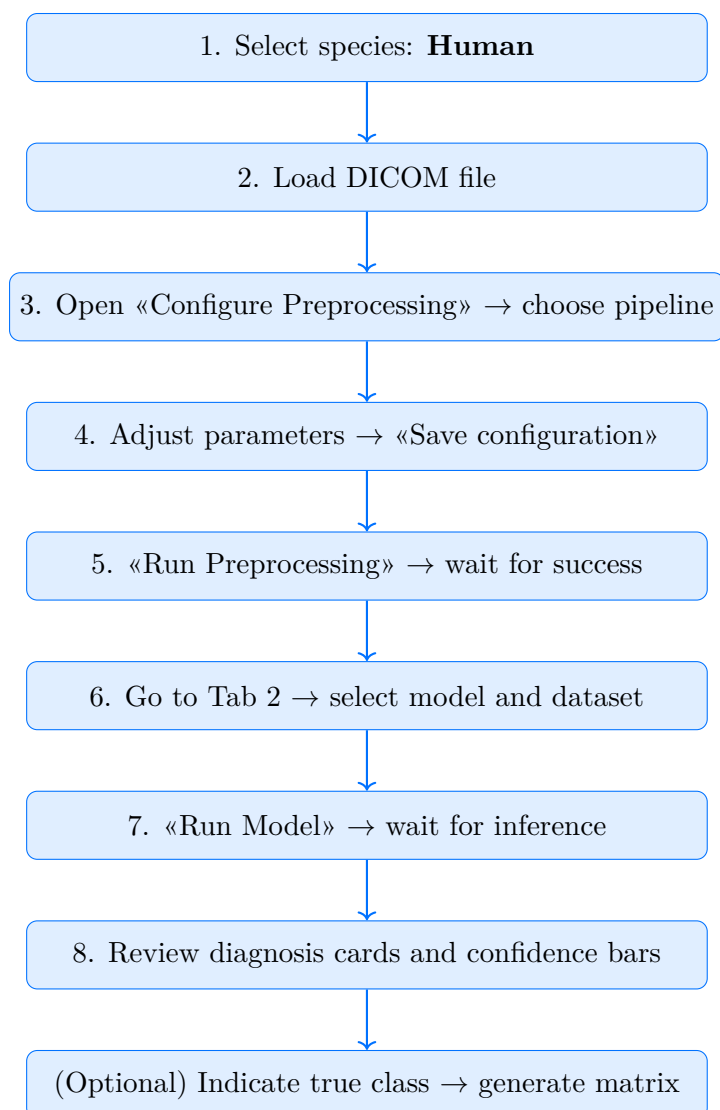
Validation and retraining

The prognosis module has the same «Validate the model» section and the same automatic retraining mechanism described in Section 5.5.1, applied to the prognosis task.

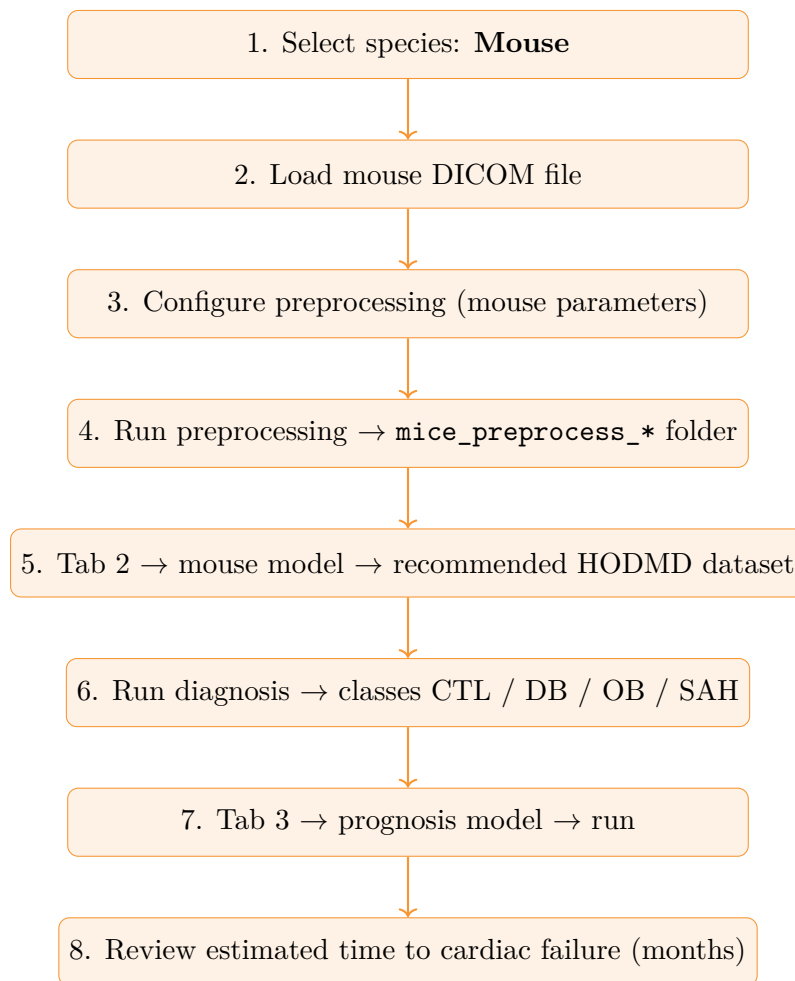
Chapter 7

Complete workflow

7.1 Diagnosis workflow — Human



7.2 Complete workflow — Mouse (Diagnosis + Prognosis)



Chapter 8

Key differences between human and mouse

Aspect	Human	Mouse
Diagnostic classes	HIPER, ISQ, NORM	CTL, DB, OB, SAH
Prognosis module	Not available	Available
Preprocessing scripts	preprocess/	mice_preprocess/
Models folder	models/	mice_models/
Output folder prefix	preprocess_*	mice_preprocess_*
Results prefix	diagnosis_*	mice_diagnosis_*
UI differentiator colour	Blue (normal)	Orange (warning)
Step 1 parameters	Inpainting switches, white lines, square crop	# snapshots, colourmap, letter recognition with thresholds
Auto-scan	Only when switching species to Human	Only when switching species to Mouse
Prognosis scale factor	—	gt_outcome_scale_factor (def. 100)

Chapter 9

Frequently asked questions and troubleshooting

Q1. The DICOM does not load and a Pillow error appears.

Install: `pip install Pillow pylibjpeg pylibjpeg-openjpeg`.

Q2. Preprocessing fails at Step 3 with «HODMD not available».

Verify that `DMDD.py` and `hosvd.py` are in the correct preprocessing folder (`preprocess/` or `mice_preprocess/`).

Q3. No models appear in the Diagnosis tab.

Verify that the `models/` (or `mice_models/`) folder contains sub-folders with at least one `checkpoint*.pth` file.

Q4. Prognosis results are very small values (~ 0.9).

Check that `input_parameters.txt` in the checkpoint contains `gt_outcome_scale_factor = 100` (or the correct value). Without this parameter the model returns the unrescaled value.

Q5. Language change does not appear in some text.

Translation is complete across all UI elements. If any text remains in Spanish it may be a system OS or Python error message, outside the scope of the application's i18n.

Q6. The window cannot be closed with the X button.

In Flet 0.84, `page.window.close()` is a coroutine; it must be called with `await`. Verify that the virtual environment has the correct version of Flet.

Q7. Confidence bar colours do not match the classes.

The species must be selected correctly *before* loading results. Change the species and reload the results folder.

Q8. The preprocessing spinner stays stuck.

Check the console where `main.py` is running: the full error is printed there even if the UI does not show it.

Glossary

Term	Definition
DICOM	<i>Digital Imaging and Communications in Medicine.</i> Standard for storing and transmitting medical images.
MAE ViT	<i>Masked Autoencoder Vision Transformer.</i> Deep learning architecture that pre-trains a vision Transformer by masking image patches and learning to reconstruct them.
SVD	<i>Singular Value Decomposition.</i> Matrix decomposition that separates signal from noise by retaining the dominant modes.
HODMD	<i>Higher-Order Dynamic Mode Decomposition.</i> Extension of classical DMD that augments the state space to capture more complex dynamics via iterative HOSVD.
Checkpoint	A <code>.pth</code> (PyTorch) file that contains the trained weights of a model.
Softmax	Function that converts raw model outputs (logits) into probabilities that sum to 1.
Scale factor	Multiplicative factor applied to the prognosis prediction to recover real units (months) from the normalised value learned during training.
Token (data)	Abbreviation identifying the type of video representation used to train or feed a model (orig, svdreconst, dmdreconst, etc.).
CTL / DB / OB / SAH	Mouse classes: Control, Diabetic, Obese, Subarachnoid Haemorrhage.
HIPER / ISQ / NORM	Human classes: Hypertrophy, Ischaemia, Normal.